OS Assignment 1 Report

학번: 2022065250

이름: 문상철

1. Design

이 과제의 목표는 xv6에서 현재 활성화된 프로세스의 상태를 표시하는 ps 유저 레벨 프로그램과 이에 대응하는 시스템 콜을 구현하는 것이다. 두 파트로 나누면 다음과 같다.

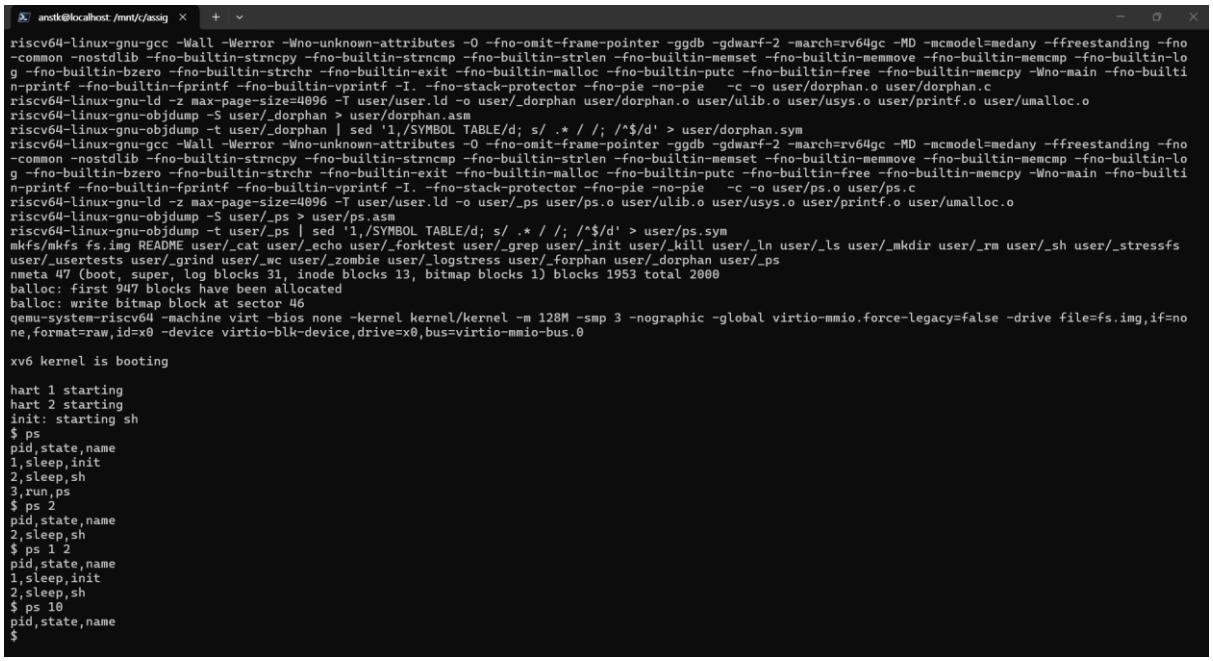
- **User Level:** ps.c 프로그램은 사용자 입력을 처리한다. 명령줄 인자를 파싱하여 모든 프로세스를 가져올지(인자 없음), 아니면 특정 프로세스(주어진 PID)를 가져올지 결정한다. 이후 조건에 맞게 ps(int pid) 시스템 콜을 호출한다.
- **Kernel Level:** kernel/proc.c의 ps(int pid) 함수는 커널의 프로세스 테이블(proc 배열)을 순회한다. UNUSED 상태의 프로세스를 필터링하고, 요청된 대상과 PID가 일치하는지 확인한다(pid != 0인 경우). 명세서 조건을 충족하기 위해 쉼표로만 구분된 엄격한 포맷으로 콘솔에 pid,state,name을 직접 출력한다.

2. Implementation

새로운 시스템 콜을 통합하기 위해 xv6 코드베이스에 다음과 같은 수정을 진행했다.

- user/ps.c **(신규 파일):** 유저 프로그램을 구현했다. 가장 먼저 pid,state,name\n 헤더를 정확히 출력한다. argc == 1 이면 ps(0)을 호출하고, 인자가 있으면 atoi()를 사용해 argv를 순회하며 특정 PID에 대해 ps(target_pid)를 호출한다.
- kernel/proc.c: 핵심 함수인 ps(int target_pid)를 추가했다. procdump()를 참고하여 proc 배열을 순회하도록 작성했다. 명세서의 요구사항을 충족하기 위해 상태 문자열 배열에서 후행 공백(예: "sleep ")을 모두 제거했다.
- kernel/sysproc.c: 래퍼 함수인 sys_ps(void)를 구현했다. 시스템 콜이 정수형 인자를 필요로 하기 때문에 유저 공간에서 안전하게 인자를 가져오기 위해 argint(0, &target_pid)를 사용했다.
- kernel/syscall.h & kernel/syscall.c: 시스템 콜 번호(#define SYS_ps 22)를 정의하고, syscalls 배열에 sys_ps 함수 포인터를 등록했다.
- kernel/defs.h & user/user.h: 커널과 유저 프로그램 모두에서 시스템 콜을 사용할 수 있도록 int ps(int); 함수 선언을 추가했다.
- user/usys.pl: usys.S에 어셈블리 스텁을 생성하기 위해 entry("ps"); 매크로를 추가했다.
- Makefile: 유저 프로그램이 xv6 이미지에 컴파일되어 포함되도록 UPROGS 목록에 $U/_ps\를 추가했다.

3. Results



```
anstk@localhost:/mnt/c/assig
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mcmodel=medany -ffreestanding -fno
-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-lo
g -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builti
n-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/dorphan.o user/dorphan.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_dorphan user/dorphan.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_dorphan > user/dorphan.asm
riscv64-linux-gnu-objdump -t user/_dorphan | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/dorphan.sym
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mcmodel=medany -ffreestanding -fno
-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-lo
g -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builti
n-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/ps.o user/ps.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_ps user/ps.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_ps > user/ps.asm
riscv64-linux-gnu-objdump -t user/_ps | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/ps.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs
user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_ps
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 947 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=no
ne,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ps
pid,state,name
1,sleep,init
2,sleep,sh
3,run,ps
$ ps 2
pid,state,name
2,sleep,sh
$ ps 1 2
pid,state,name
1,sleep,init
2,sleep,sh
$ ps 10
pid,state,name
$
```

1. $ ps: 헤더와 모든 활성화된 프로세스(init, sh, ps)의 세부 정보를 성공적으로 출력한다.
2. $ ps 2: 지정한 PID 2번(sh) 프로세스만 올바르게 필터링하여 출력한다.
3. $ ps 1 2: 다중 인자를 정확하게 처리하여 PID 1과 PID 2의 정보를 순차적으로 출력한다.
4. $ ps 10: 존재하지 않는 프로세스를 지정할 경우 유효하지 않은 데이터를 출력하지 않으며, 지정된 프로세스가 없으므로 시스템 콜이 1을 반환한다.

4. Troubleshooting

구현 중 별 다른 문제는 발생하지 않았으나, 잠재적인 문제를 예방하기 위해 다음 사항들을 염두에 두고 진행했다.

- **엄격한 포맷팅 준수:** 과제 명세서에 출력 포맷이 조금이라도 어긋나면 0점 처리될 수 있다고 명시되어 있었고 기존 procdump() 코드는 정렬을 위해 공백을 포함하고 있었다. 그래서 감점 방지를 위해 상태 문자열 배열에서 모든 후행 공백을 명시적으로 제거했다.

- **안전한 인자 전달:** 처음에는 매개변수를 직접 전달하는 것을 고려했다. 하지만 유저와 커널 공간 사이의 컨텍스트 경계를 인지하고, kernel/sysproc.c에서 엄격하게 argint() 함수를 활용했다. 이는 시스템 콜 인자를 검증하고 포인터나 값이 유효한 주소 공간을 참조하도록 보장하여, 잘못된 유저 입력으로 인한 커널 패닉을 방지한다.

5. System Call Execution Flow

ps 시스템 콜의 전체 실행 경로는 다음과 같다:

1. **유저 호출:** 사용자가 $ ps를 실행하면 user/ps.c에서 ps() 함수가 호출된다.
2. **어셈블리 스텁:** 호출은 usys.pl에 의해 생성된 usys.S의 어셈블리 코드로 넘어간다. a7 레지스터에 시스템 콜 번호(SYS_ps)를 로드하고 ecall 명령어를 실행한다.
3. **커널 트랩:** ecall은 CPU를 슈퍼바이저 모드로 전환한다. 실행 흐름은 유저 레지스터를 저장하는 trampoline.S의 uservec으로 넘어가고, 이어서 trap.c의 usertrap()으로 진입한다.
4. **시스템 콜 디스패치:** usertrap()은 흐름을 kernel/syscall.c의 syscall()로 라우팅한다. syscall()은 a7 값을 읽어 syscalls 배열의 인덱스로 사용하고 래퍼 함수인 sys_ps()를 호출한다.
5. **커널 실행:** sysproc.c에 위치한 sys_ps()는 argint(0, &target_pid)를 호출하여 트랩 프레임에서 인자를 읽어온다. 그 후 proc.c에 있는 실제 ps(target_pid)를 호출하여 프로세스 테이블을 순회하고 출력을 진행한다.
6. **반환:** ps()는 정수(0 or 1)를 반환한다. syscall()은 이 반환값을 트랩프레임의 a0 레지스터에 저장한다. 마지막으로 trampoline.S의 userret()이 유저 레지스터를 복원하고 sret 명령어를 실행하여 유저 프로그램으로 제어권을 반환한다.